

# SUNUM 11

AD: MEHMET EMRE ŐENGÖZ

ÖÖRENCİ NO: 2322705065

DERS: MAKİNE ÖÖRENMESİNE GİRİŐ



# GİRİŞ

## TEMEL KAVRAMLAR

- 2.1. Autoencoders (Otomatik Kodlayıcılar)
- 2.2. Variational Autoencoders (VAE)
- 2.3. Generative Adversarial Networks (GAN)
- 2.4. Super Resolution (Süper Çözünürlük)
- 2.5. Latent Space (Gizli Uzay) Kavramı

## YÖNTEM VE MODEL MİMARİSİ

## UYGULAMA ALANLARI

## MİNİ PROJE: AUTOENCODER İLE GÖRÜNTÜ YENİDEN YAPILANDIRMA

- 5.1. Kullanılan Veri Seti (MNIST)
- 5.2. Model Yapısı ve Eğitim Süreci
- 5.3. Elde Edilen Sonuçlar ve Program Çıktıları

## SONUÇ

# KAYNAKÇA

## 1. GİRİŞ

Makine öğrenmesi dünyasında veriyi sadece sınıflandırmak değil, verinin özünü anlayarak yeni veriler üretmek büyük bir öneme sahiptir. Bu rapor kapsamında, denetimsiz öğrenmenin en güçlü araçlarından biri olan Autoencoder yapıları ve üretken modeller (Generative Models) ele alınmıştır. Çalışmanın amacı, karmaşık verilerin düşük boyutlu bir uzayda (Latent Space) nasıl temsil edildiğini ve bu temsil üzerinden verinin nasıl yeniden inşa edildiğini teorik ve uygulamalı olarak göstermektir.

## 2. TEMEL KAVRAMLAR

### 2.1. Autoencoders (AE)

Autoencoder'lar, giriş verisini önce sıkıştıran (Encoder) ve ardından bu sıkıştırılmış bilgiden orijinal veriyi geri üretmeye çalışan (Decoder) yapay sinir ağlarıdır. Modelin amacı, veriyi "kopyalamak" değil, verinin en önemli özelliklerini (features) öğrenmektir.

### 2.2. Variational Autoencoders (VAE)

Standart Autoencoder'lardan farklı olarak VAE'ler, gizli uzayı (latent space) sabit bir nokta olarak değil, bir olasılık dağılımı olarak modeller. Bu durum, modelin sadece veriyi yeniden inşa etmesini değil, aynı zamanda bu dağılımdan örnekleme yaparak tamamen yeni veriler üretmesini sağlar.

### 2.3. Generative Adversarial Networks (GAN)

GAN, iki farklı sinir ağının birbiriyle yarıştığı bir yapıdır.

**Generator (Üretici)** sahte veri üretirken, **Discriminator (Ayırt**

**Edici)** verinin gerçek mi yoksa sahte mi olduğunu anlamaya çalışır. Bu rekabet, üretilen verilerin gerçeğe en yakın seviyeye ulaşmasını sağlar.

## 2.4. Super Resolution

Super Resolution, düşük çözünürlüklü görüntülerin derin öğrenme yöntemleri kullanılarak yüksek çözünürlüklü ve net hale getirilmesi işlemidir. AI modelleri, pikseller arasındaki eksik bilgileri tahmin ederek görüntüyü zenginleştirir.

## 2.5. Latent Space (Gizli Uzay) Kavramı

Latent Space, bir verinin en temel özelliklerinin temsil edildiği, boyutça küçültülmüş gizli bir katmandır. Örneğin, bir insan yüzü binlerce pikselden oluşur; ancak latent space'te bu yüz "gülümseme miktarı", "kaş yapısı" veya "ten rengi" gibi birkaç temel sayısal değerle temsil edilebilir.

---

## 3. YÖNTEM VE MODEL MİMARİSİ

Bu çalışmada gerçekleştirilen uygulama, "Bottleneck" (Darboğaz) mimarisine sahip bir Autoencoder yapısıdır. Model şu adımları izler:

1. **Giriş Katmanı:** 28x28 boyutundaki MNIST görselleri düzleştirilerek 784 birimlik bir vektöre dönüştürülür.
2. **Encoder:** Veri, 128 nöronlu bir gizli katmandan geçirilerek öznitelik çıkarımı yapılır.
3. **Bottleneck (Latent Space):** Veri 32 birime sıkıştırılır. Bu, verinin %95'ten fazla oranda sıkıştırılması anlamına gelir.
4. **Decoder:** 32 birimlik bilgi, tekrar 128 nöronlu katmana ve ardından 784 birimlik orijinal boyutuna geri açılır.

## 5. MİNİ PROJE: AUTOENCODER İLE GÖRÜNTÜ YENİDEN YAPILANDIRMA

### 5.1. Kullanılan Veri Seti

Projelerde el yazısı rakamlardan oluşan **MNIST** veri seti kullanılmıştır. Bu veri seti, makine öğrenmesi algoritmalarını test etmek için dünya çapında standart kabul edilen bir veri kümesidir.

### 5.2. Model Yapısı ve Eğitim Süreci

Model, Python dili ve TensorFlow/Keras kütüphaneleri kullanılarak geliştirilmiştir. Kayıp fonksiyonu olarak **Mean Squared Error (MSE)**, optimizasyon algoritması olarak ise **Adam** tercih edilmiştir. Model 10 epoch boyunca eğitilmiştir.

```
Autoencoder ile Görüntü Yeniden Yapılandırma

[1]
> import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, losses
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model

Adım 1: Verilerin Yükleme ve Normalize Edilmesi

[2]
> (x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = x_train.reshape((x_train.shape[0], -1))
x_test = x_test.reshape((x_test.shape[0], -1))

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 1s 0us/step

Adım 2: Model Mimarısının Kurulması (Encoder & Decoder)

[3]
> latent_dim = 32

class Autoencoder(Model):
    def __init__(self, latent_dim):
        super(Autoencoder, self).__init__()
        self.latent_dim = latent_dim
        self.encoder = tf.keras.Sequential([
            layers.Flatten(),
            layers.Dense(128, activation='relu'),
            layers.Dense(latent_dim, activation='relu'),
        ])
        self.decoder = tf.keras.Sequential([
            layers.Dense(128, activation='relu'),
            layers.Dense(784, activation='sigmoid'),
            layers.Reshape((28, 28))
        ])

    def call(self, x):
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
        return decoded

autoencoder = Autoencoder(latent_dim)
autoencoder.compile(optimizer='adam', loss=losses.MeanSquaredError())

Adım 3: Eğitimi ve Görselleştirme

[4]
> autoencoder.fit(x_train, x_train, epochs=10, shuffle=True, validation_data=(x_test, x_test))

# Test verisiyle deneme sonucu
decoded_imgs = autoencoder.predict(x_test)

# Orjinal resim
n = 10
plt.figure(figsize=(20, 4))
for i in range(n):
    # Orjinal resim
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

    # Rekonstrüktör resim
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

plt.show()

Epoch 1/10
1875/1875 --- 9s 4ms/step - loss: 0.0252 - val_loss: 0.0138
Epoch 2/10
1875/1875 --- 8s 4ms/step - loss: 0.0125 - val_loss: 0.0138
Epoch 3/10
1875/1875 --- 7s 4ms/step - loss: 0.0107 - val_loss: 0.0108
Epoch 4/10
1875/1875 --- 7s 4ms/step - loss: 0.0099 - val_loss: 0.0093
Epoch 5/10
1875/1875 --- 7s 4ms/step - loss: 0.0093 - val_loss: 0.0088
Epoch 6/10
1875/1875 --- 6s 3ms/step - loss: 0.0088 - val_loss: 0.0085
Epoch 7/10
1875/1875 --- 6s 4ms/step - loss: 0.0085 - val_loss: 0.0081
Epoch 8/10
1875/1875 --- 7s 4ms/step - loss: 0.0082 - val_loss: 0.0079
Epoch 9/10
1875/1875 --- 7s 4ms/step - loss: 0.0080 - val_loss: 0.0077
Epoch 10/10
1875/1875 --- 7s 4ms/step - loss: 0.0078 - val_loss: 0.0075
31.1s/10s
```

### 5.3. Elde Edilen Sonular ve Program ıktıları

Eđitim sonucunda modelin kayıp (loss) deęerinin istikrarlı bir řekilde dūřtūęū gōzlemlenmiřtir. Test veri seti zerindeki sonular, modelin pikselleri yksek doęrulukla geri inřa edebildięini kanıtlamıřtır.



### 6. SONU

Bu alıřma kapsamında, derin ęrenmenin retken modelleri teorik olarak incelenmiř ve bir Autoencoder uygulaması bařarıyla gerekleřtirilmiřtir. Latent Space kavramının, verinin boyutunu indirgerken nemli bilgileri nasıl koruduęu somut bir řekilde gōzlemlenmiřtir. Modelin bařarısı, ileride grlt temizleme (denoising) ve veri retme gibi daha karmařık gōrevlerde kullanılabileceęini gōstermektedir.

### 7. KAYNAKA (IEEE Formatı)

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [2] F. Chollet, *Deep Learning with Python*. Manning Publications, 2017. [3] TensorFlow Documentation, "Autoencoders,"

[Online]. Available:

<https://www.tensorflow.org/tutorials/generative/autoencoder>